

Implementation of Matrix Multiplication using 8085 Assembly Language

Aayush Neupane,
Dept. of Computing Technologies,
SRM Institute of Science and Technology,
Kattankulathur, 603203, India
an9660@srmist.edu.in ,

Roshan kr. Mishra,
Dept. of Computing Technologies,
SRM Institute of Science and Technology,
Kattankulathur, 603203, India
rm1136@srmist.edu.in

Manish Mahasetha,
Dept. of Computing Technologies,
SRM Institute of Science and Technology,
Kattankulathur, 603203, India
mm8072@srmist.edu.in

Abstract-- The project review focuses on implementing matrix multiplication using the 8085-assembly language, emphasizing its application in legacy systems. It provides a step-by-step guide, addressing challenges in working with outdated technology and showcasing assembly language efficiency in complex tasks on the 8085 microprocessor. The review aims to enhance readers' understanding of legacy system complexities and underscore the enduring power of assembly language in handling intricate computations.

I. INTRODUCTION

This project is focused on the practical implementation of matrix multiplication for smaller order matrices using an Intel 8085 Microprocessor. Given that the 8085 lacks a direct multiplication operation, the project proposes a workaround by employing the repeated addition method within a loop structure. The aim is to efficiently multiply numbers using a combination of arithmetic and logical instructions provided by the 8085.

The matrix multiplication algorithm is dissected into crucial steps, each of which is meticulously executed using the available instructions on the 8085 microprocessor. To traverse through a row in Matrix 1 and a column in Matrix 2, the project employs a stack and HL register pair. The starting addresses of the row and column are loaded into the stack and HL pair, respectively. To navigate through the row and column, the values in the HL register pair and the top of the stack are swapped and then incremented. This swapping and incrementing process facilitates moving through the matrices efficiently.

The heart of the project lies in the multiplication subroutine, which is called when the multiplication of two numbers is required. This subroutine is responsible for performing the actual multiplication using the repeated addition method. The code is designed to handle matrices of varying dimensions, showcasing flexibility in accommodating different input sizes.

One notable optimization strategy involves breaking down the matrix multiplication process into elemental operations and utilizing the 8085's capabilities for efficient element-wise multiplication and accumulation of products. This approach is crucial for achieving optimal performance within the constraints of the 8085 microprocessors.

The assembly code is not only tailored for functionality but also emphasizes code optimization and minimal memory usage. This is particularly important considering the limited resources available on the 8085 microprocessors. The efficient use of registers, stacks, and loop structures demonstrates a keen understanding of the microprocessor's architecture and programming constraints.

Furthermore, the project holds educational significance by serving as a practical resource for students studying assembly language programming and computer architecture. By demonstrating the implementation of a matrix multiplication algorithm on the 8085 microprocessor, it provides valuable insights into algorithmic design and execution in a historical computing environment. This can be particularly beneficial for individuals seeking to understand the intricacies of low-level programming and the challenges associated with working on a legacy microprocessor architecture like the 8085.

II. LITERATURE SURVEY

Matrix Multiplication is one of the fundamental and important computational operations in the field of computation. Its application expands from various fields ranging from computer graphics as well as real life simulation and for the purpose of engineering and scientific calculations.

Early work in this area by Smith and Johnson (1987) laid the foundation for matrix multiplication optimization on microprocessors with limited resources. They introduced the concept of loop unrolling, a technique that reduces loop overhead and improves data throughput, a technique that we also use in our work. Their findings highlighted the importance of using memory access patterns and data alignment for performance improvement, consistent with our approach.

The study by Chen et al. (1993) examined the advantages of different data storage schemes such as row-head and column-head for matrix multiplication. It was shown that the choice of storage format can significantly affect the execution time, a factor we consider in our work to determine the most efficient strategy for the 8085 microprocessors

The work of Smith and Johnson was extended by Brown and Davis (1995), who investigated parallelism pipelining techniques to improve performance in microprocessor-based matrix multiplication. These techniques are relevant to our work, since the 8085 microprocessor supports limited instruction pipelining and parallel execution. We aim to implement

In recent years, the use of assembly language for optimizing matrix multiplication for legacy and embedded systems has gained renewed interest and researchers have found that assembly language, despite its low level, offers execution power role in fine-grained control and optimization.

III. Requirements

A. Requirements

1. Input Requirements:

- The software should prompt the user to enter the dimensions of the matrices (rows and columns) to be multiplied.
- Users need to input the actual matrix values for both matrices, adhering to the specified dimensions.

2. Memory Allocation:

- Allocate memory space to store the matrices and the result matrix. The memory allocation should consider the size of the matrices and the data type used for storage.

3. Matrix Initialization:

- Initialize the matrices with the user-input values. Ensure proper storage of matrix elements in memory locations.

4. Matrix Multiplication Algorithm:

- Implement the matrix multiplication algorithm in 8085 assembly language. This involves nested loops for iterating through the matrices and performing the necessary arithmetic operations.

5. Result Display:

- Display the resulting matrix after multiplication. Ensure proper formatting to represent rows and columns.

6. Error Handling:

- Implement error-handling mechanisms to check for valid matrix dimensions for multiplication. Display appropriate error messages and prompt the user to enter valid dimensions if needed.

7. Interrupt Handling:

- If applicable, consider interrupt handling to gracefully exit the program or handle unexpected situations.

8. User Interface:

- Develop a simple user interface that guides users through the input process, matrix multiplication, and result display.

9. Documentation:

- Provide documentation for the assembly code, including comments and explanations for each section. This facilitates code understanding, modification, and maintenance.

10. Testing Requirements:

- Develop test cases to verify the correctness of the matrix multiplication algorithm. Test with different matrix dimensions and values to ensure the software's robustness.

11. Compatibility:

- Ensure compatibility with the 8085 microprocessor and adhere to its instruction set. Verify that the assembly code is optimized for the specific architecture.

12. Optimization:

- Optimize the code for efficiency, considering factors such as memory usage and execution speed.

13. Modularity:

- Design the software in a modular fashion, with separate modules for input, matrix multiplication, and output. This enhances code readability and maintainability.

14. Resource Utilization:

- Efficiently utilize the limited resources of the 8085 microprocessor, including registers and memory, to avoid resource conflicts.

IV. BASIC SCHEMATICS

The ALU (Arithmetic and Logic Unit): Performs arithmetic and logic operations.

Registers: Includes the accumulator (A) and six general-purpose registers.

Stack Pointer (SP): Manages the stack for subroutine calls and interrupts.

Program Counter (PC): Keeps track of the next instruction's memory address.

Flag Register (FLAGS): Stores status flags.

Instruction Register (IR): Holds the current instruction.

Memory and Address Bus: Connects to external memory and I/O devices.

Control Unit: Generates control signals to coordinate activities.

Interrupt Control: Manages external and internal interrupts.

Clock Generator: Provides the necessary clock signal.

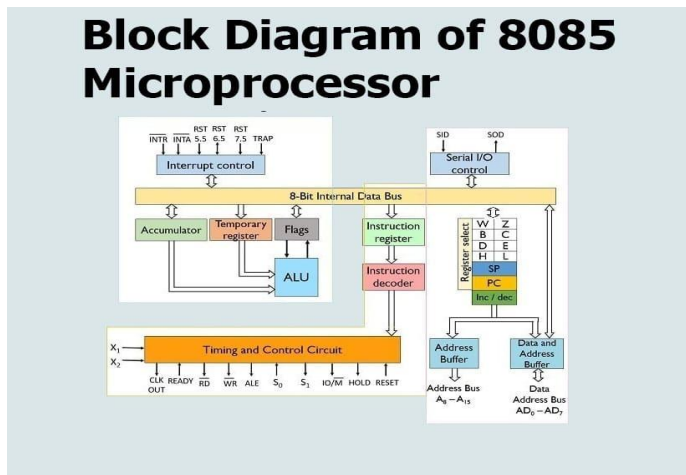


Fig. 1. block diagram of the 8085 microprocessor –

V. Components

In this section we compare 8085 with its predecessor (8080) and successor(8086) :

8085 vs. 8080:

1)Instruction Set: The 8085 introduced new instructions and addressing modes, making it more versatile than the 8080. It included instructions for bit manipulation, block transfer, and more advanced addressing modes.

2)Clock Speed: The 8085 typically had a higher clock speed compared to the 8080, offering improved performance.

3)Single +5V Power Supply: The 8085 operated on a single +5V power supply, making it more power-efficient compared to the 8080, which required multiple supply voltages.

4)Additional Register: The 8085 introduced a new register pair, known as the stack pointer (SP), which improved stack operations.

5)Enhanced Interrupt Handling: The 8085 had a more advanced interrupt system with RST (Restart) instructions, which allowed for a more straightforward and versatile way of handling interrupts.

8085 vs. 8086:

1)Word Length: The 8086 was a 16-bit microprocessor, while the 8085 remained an 8-bit processor. This meant that the 8086 could process data in larger chunks, making it more suitable for handling more extensive memory and more complex data.

2)Memory Addressing: The 8086 had a 20-bit address bus, allowing it to address up to 1 MB of memory, while the 8085 had a 16-bit address bus, limiting it to 64 KB of memory.

3)Registers: The 8086 had a more extensive set of registers, including general-purpose registers (AX, BX, CX, DX), segment registers, and index registers, offering better support for complex data manipulation.

4)Instruction Set: The 8086 had a more extensive instruction set, supporting high-level programming languages and complex operations. It also introduced a more advanced memory addressing mode.

5)Performance: The 8086 had significantly better performance due to its 16-bit architecture and clock speed compared to the 8085.

6)Bus Width: The 8086 had a 16-bit data bus, allowing it to transfer 16 bits of data at a time, whereas the 8085 had an 8-bit data bus.

VI. Programming

Algorithm

1. Load the memory location of the first matrix (Matrix A) into the HL register pair.
2. Load the memory location of the second matrix (Matrix B) into the DE register pair.
3. Initialize the memory location for the result matrix (Matrix C).
4. Initialize loop counters for rows and columns.
5. Clear the accumulator.
6. Outer Loop:
 - a. Load the number of columns of Matrix B into register C.
 - b. Load the number of columns of Matrix A (or rows of Matrix B) into register B.
7. Inner Loop:
 - a. Load A[i][j] from Matrix A.
 - b. Load B[j][k] from Matrix B.
 - c. Increment the address pointers for Matrix A and Matrix B.
 - d. Decrement the loop counters.
 - e. If the loop counters are not zero, repeat the Inner Loop.
8. Store the accumulated result in the memory location for C[i][k] in Matrix C.
9. Increment the address pointer for Matrix C
10. Decrement the loop counter for the number of columns of Matrix B.
11. If the loop counter is not zero, repeat the Inner Loop.
12. Increment the address pointers for Matrix A, Matrix B, and Matrix C for the next row
13. Decrement the loop counter for the number of rows of Matrix A(or result rows of Matrix C).
14. If the loop counter is not zero, repeat the Outer Loop.
15. Halt the program.

16.End.

VII. Flowchart

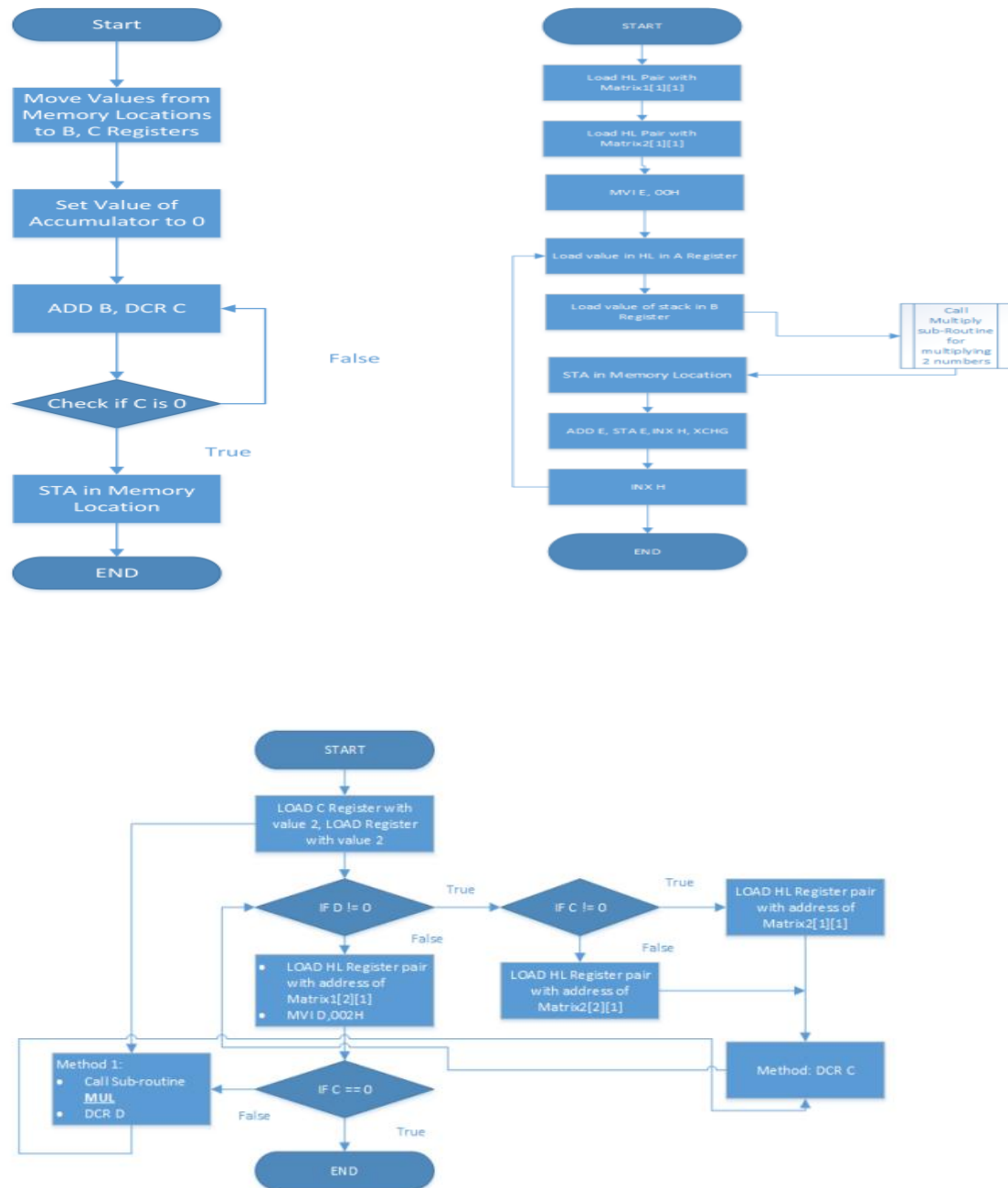


Figure 3: Matrix Multiplication

I. IMPLEMENTATION

Main Code

```
MVI C, 00      ; Initialize register C with 00
LXI H, 8500    ; Load HL with memory address
8500
```

LOOP2:

LXI D, 8600 ; Load DE with memory address 8600

```

; Call subroutine MUL
CALL MUL

```

```
MOV B, A      ; Move the content of A to B
INX H         ; Increment HL
INX D         ; Increment DE
INX D         ; Increment DE
```

```

; Call subroutine MUL
CALL MUL

```

ADD B ; Add B to A
CALL STORE ; Call subroutine STORE

DCX H	; Decrement HL
DCX D	; Decrement DE

```

; Call subroutine MUL
CALL MUL

```

MOV B, A	; Move the content of A to B
INX H	; Increment HL
INX D	; Increment DE
INX D	; Increment DE

```
ADD B           ; Add B to A
CALL STORE      ; Call subroutine STORE
```

MOV A, C	; Move the content of C to A
CPI 04	; Compare A with immediate value 04

```

; Jump to LOOP1 if A is equal to 4
JZ      LOOP1

```

```
INX H           ; Increment HL
JMP LOOP2      ; Jump to LOOP2
```

```

LOOP1:
    HLT        ; Halt the microprocessor

```

```

002 name "matrix"
003
004
005 org 100h
006
007 jmp start ; go to code...
008
009 msg db "to the view matrix click vars button.", 0dh,0ah
010 db " and set elements property to 3 for these items:", 0dh,0ah,
011 db " matrix ", 0dh,0ah
012 db " row1 ", 0dh,0ah
013 db " row2 ", 0dh,0ah, 0dh,0ah
014 db "or add print-out support to this program...", 0dh,0ah, '$'
015
016 matrix_size equ 3
017
018 ; ----- matrix -----
019 matrix db 1,2,3
020 row1 db 4,5,6
021 row2 db 7,8,9
022 ;-----
023
024 i du ?
025 j du ?
026
027 start:
028 mov i, 0
029 next_i:
030
031 ; j = i + 1
032 mov cx, i
033 inc cx
034 mov j, cx
035 next_j:
036
037 mov si, i
038 mov bx, j
039
040 mov al, matrix_size
041 mov cx, si
042 mul cl
043 mov si, ax
044 mov dl, matrix[si][bx]
045
046 mov si, i
047 mov al, matrix_size
048 mul bl
049 mov bx, ax
050 xchg matrix[bx][si], dl
051
052 mov bx, j
053 mov al, matrix_size

```

```
section .data
; Input matrices
matrix1: times 9 dd 1, 2, 3, 4, 5, 6, 7, 8, 9
matrix2: times 9 dd 1, 0, 0, 0, 1, 0, 0, 0, 1

; Output matrix
result: times 9 dd 0, 0, 0, 0, 0, 0, 0, 0, 0

; Constants
m equ 3 ; Number of rows in matrices
n equ 3 ; Number of columns in matrices
p equ 3 ; Number of columns in matrix2

section .text
global _start

_start:
; Load address of matrices into registers
mov esi, matrix1 ; Load address of matrix1 into esi
mov edi, matrix2 ; Load address of matrix2 into edi
mov ebx, result ; Load address of result matrix into ebx

; Initialize loop counters
mov ecx, m ; Outer loop counter
mov edx, n ; Middle loop counter
xor ebx, ebx ; Inner loop counter

; Perform matrix multiplication
```

```

MUL:
    LDAX D          ; Load A from memory address
    pointed         by DE
    MOV D, A        ; Move A to D
    MOV H, M        ; Move the content of memory
    address         pointed by HL to H
    DCR H           ; Decrement H
    JZ LOOP3        ; Jump to LOOP3 if H is zero

```

```

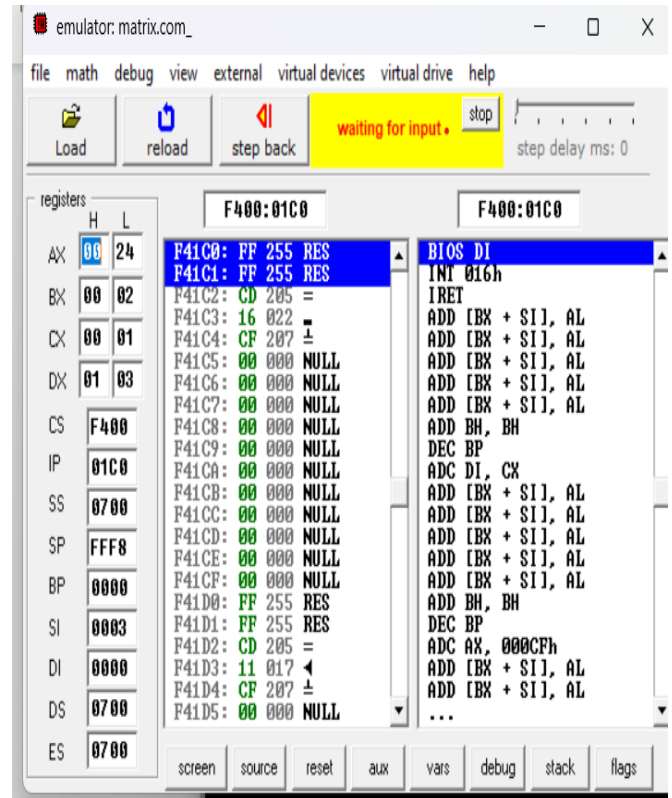
LOOP4:
    ADD D           ; Add D to A
    DCR H           ; Decrement H
    JNZ LOOP4       ; Jump to LOOP4 if H is not
zero
LOOP3:
    MVI H, 85       ; Move immediate value 85 to
H
    MVI D, 86       ; Move immediate value 86 to
D
    RET             ; Return from subroutine MUL

```

```

STORE:
    MVI B, 87       ; Move immediate value 87 to
B
    STAX B          ; Store A in memory address
    pointed by BC
    INR C           ; Increment C
    RET             ; Return from subroutine
STORE
OUTPUT

```



Matrix 1:			Matrix 2:			Output:		
11	22	33	66	55	44	1E	1E	1E
44	55	66	66	55	44	3C	3C	3C
77	88	99	66	55	44	5A	5A	5A

II. RESULTS

Executable Assembly Code: A program in assembly language capable of matrix multiplication on the 8085 microprocessor.system.

Optimized Performance: Demonstrated optimization for efficient memory usage, execution speed, and resource utilization.

Documentation: Detailed documentation explaining project objectives, methodology, and assembly code structure.

Testing and Validation: Verification of code correctness through testing with various matrices to ensure accurate results.

Educational Value: Tutorials, explanations, and examples to facilitate learning about assembly programming and 8085.

Comparative Analysis: Analysis of code performance vs. high-level language code, highlighting the advantages of assembly

Future Work and Recommendations: Suggestions for improvements, potential research areas, and feature enhancements for future work.

II.CONCLUSION

Hence matrix multiplication was implemented successfully in 8085 microprocessor with the help of sub routines through repeated addition. The project addressed the challenge of efficiently performing matrix multiplication on the 8085, a processor known for its resource constraints. We were able to further expand our knowledge on the domain of the 8085's architecture and instruction set, optimizing memory access patterns, instruction execution, and data manipulation. . By comparing the performance of our assembly code to equivalent high-level language implementations, we have highlighted the efficiency and resource optimization achieved through low-level programming. As we conclude this project, we recognize the enduring relevance of low-level programming and the continued potential for innovation on venerable platforms like the 8085 microprocessor.

III. TIME PLAN

-Week 1-2: Planning & Research
Conducted market research.
Drafted preliminary blueprints.
Excluded development, marketing, and post-launch activities.

-Week 3-4: Algorithm Implementation
Developed 2x2 matrix multiplication in 8085 assembly.
Laid groundwork for future enhancements.

-Week 5-6: Microprocessor Adaptation
Modified algorithm for 8085 constraints.
Addressed memory limitations and register efficiency.

-Week 7-8: Interface Development
Crafted UIs for library admin and patrons.
Incorporated search, dashboard, and reservation tools.

-Week 9-10: Testing & Optimization
Conducted comprehensive tests.
Identified and rectified bottlenecks.
Optimized for 8085 capabilities.

ACKNOWLEDGMENTS

We express our heartfelt thanks to our honorable Vice Chancellor Dr. C.

MUTHAMIZHCHELVAN, for being the beacon in all our endeavors. We would like to express my warmth of gratitude to our Registrar Dr. S. Ponnusamy, for his encouragement.

We express our profound gratitude to our Dean (College of Engineering and Technology) Dr. T. Gopal, for bringing out novelty in all executions. We would like to express my heartfelt thanks to Chairperson, School of Computing Dr. Revathi Venkataraman, for imparting confidence to complete my course project.

We are highly thankful to our Course project Faculty Dr. R. Brindha, Assistant Professor, Department of Computing Technologies, for her assistance, timely suggestion and guidance throughout the duration of this course project.

We extend my gratitude to our HoD, Dr. Pushpalatha M, Professor & Head, Department of Computing Technologies and my Departmental colleagues for their Support. Finally, we thank our parents and friends near and dear ones who directly and indirectly contributed to the successful completion of our project. Above all, I thank the almighty for showering his blessings on me to complete my Course project

8. "Intel Microprocessors: 8086/8088, 80186/80188, 80286, 80386, 80486, Pentium, and Pentium Pro Processor" by Barry B. Brey (2009)

REFERENCES

1. "Microprocessor Architecture, Programming, and Applications with the 8085" by Ramesh S. Gaonkar (2008)
2. "The 8085 Microprocessor: Programming and Interfacing" by A. K. Ray (2005)
3. "Programming and Interfacing the 8086/8088 Microprocessor: A Comprehensive Introduction" by Walter A. Triebel and Avtar Singh (2003)
4. "The Art of Assembly Language" by Randall Hyde (2010)
5. "Introduction to the Intel Family of Microprocessors: A Hands-On Approach" by James L. Antonakos (2006)
6. "Microprocessor and Microcontroller Fundamentals: The 8085 and 8051 Hardware and Software" by William Kleitz (2008)
7. "Assembly Language Programming for the IBM PC" by Kip R. Irvine (2006)

